



VIT[®]

BHOPAL
www.vitbhopal.ac.in

PROGRAMMING IN JAVA

COURSECODE: CSE2006

NAME: TUSHITA PATHAK

REGISTRATION NO. : 25BAI10339

TRAVEL GUIDE AND EXPENSE MANAGER

1. Executive Summary

The project is a command-line one-way travel planning and expense-management MVP. It asks for an origin, destination, transport mode, distance, number of travellers, and number of nights. It then calculates per-person budget ranges for Low, Medium, and High spending styles, lets the user select a planning band, creates a segment-wise travel budget, displays generic and limited city-specific suggestions, records actual group expenses, produces category-level alerts, and gives cheaper alternatives when spending exceeds a planned category range. Finally, it compares planned and actual spending and reports whether the trip stayed within the selected band.

2. Problem Statement

Travel planning often requires a user to combine transportation, food, accommodation, activities, and local transport costs. The implemented MVP addresses this by combining route/transport inputs with predefined budget assumptions and an expense tracker. Instead of returning only one total amount, it breaks the trip into segments and spending categories, allowing the traveller to monitor expenses during the journey.

3. Proposed Solution

- Collect basic one-way trip information through a console interface.
- Use a stored route distance when a supported city pair is available; otherwise ask the user for an approximate distance.
- Estimate transportation cost according to the selected mode and number of travellers.
- Calculate Low, Medium, and High per-person budget bands.
- Allow the user to choose the budget band against which the trip will be planned.
- Generate segments such as pre-departure, travel leg, arrival day, full destination days, and final day.
- Track actual group spending category by category.
- Compare actual spending with the planned range and issue OVER, TRENDING_HIGH, or ON_TRACK feedback.
- Offer lower-cost suggestions for the next segment when a category is over budget.
- Produce a final group and per-person summary.

4. Objectives

- Provide a beginner-friendly travel budgeting workflow.
- Convert a broad travel plan into understandable spending categories.
- Support group travel by distinguishing per-person and group amounts.
- Use ranges rather than presenting estimates as exact prices.
- Monitor spending while the trip is being entered into the system.
- Give corrective feedback when actual spending exceeds the selected plan.
- Keep the MVP simple enough to run directly in a Java command-line environment.

7. System Workflow

Step 1 : Route and transport: origin → destination → transport mode → distance.

Step 2 : Travellers and budget band: travellers + nights → trip duration → Low/Medium/High ranges → selected tier.

Step 3 : Segment plan: selected tier → segment-by-segment category ranges and suggestions.

Step 4 : Expense tracking: actual group spending is entered for every category in every segment.

Step 5 : Feedback and summary: compare actuals with planned ranges → alerts/cheaper alternatives → final totals.

9. Budget Calculation Logic

The code uses predefined assumptions rather than live market prices. Transport has a mode-specific rate per kilometre, speed, overhead, and a flag indicating whether the cost is shared across travellers. Budget tiers apply a transport factor and fixed per-day/per-night assumptions for food, accommodation, activities, and local transport. A 12% buffer is then applied to the total point estimate, and the resulting point estimate is converted into a $\pm 15\%$ range.

Conceptual calculation: Total per-person estimate = (transport + food + accommodation + activities + local transport) $\times$ 1.12. Budget range = point estimate $\pm 15\%$.

The code also estimates transit meals from travel duration using approximately one meal per six hours, with at least one meal. The trip duration is nights + 1 day.

12. Route and Recommendation Data

The route catalogue currently contains these city pairs and distances: Delhi–Jaipur 280 km, Delhi–Manali 535 km, Delhi–Agra 233 km, Mumbai–Goa 590 km, Bhopal–Indore 195 km, Bhopal–Goa 1220 km, Bhopal–Delhi 780 km, and Bengaluru–Goa 560 km. Route keys are normalized so the pair can be found regardless of order. If the pair is not found, the user enters an approximate distance.

The suggestion catalogue contains generic food, accommodation, activity, and local-transport suggestions, plus two Goa-specific food entries. Suggestions are selected by city/category/tier and sorted by per-person cost.

13. Expense Tracking and Intelligent Feedback

The tracker stores actual group spending separately for each segment and category. For every planned category, the user can enter an actual amount or press Enter to use the planned midpoint. The system compares actual spending with the planned range.

Alert	Condition in code	Meaning
OVER	Actual > planned maximum	Spending has exceeded the planned range; the code can show cheaper options for a following segment.
TRENDING_HIGH	Actual > 105% of planned midpoint but still within maximum	Spending is above the midpoint and should be monitored.
ON_TRACK	Otherwise	Spending is within the intended tracking threshold.

15. Object-Oriented Design Used

- Encapsulation: domain behavior is distributed among focused classes such as BudgetEngine, ExpenseTracker, SegmentPlanner, and InputReader.
- Enums: Category, TransportMode, BudgetTier, and AlertLevel constrain the set of supported states.
- Composition: Trip contains trip information; Segment contains category ranges; ExpenseTracker works with a Trip and a list of Segments.
- Collections: ArrayList, EnumMap, LinkedHashMap, and List are used to organize runtime data.
- Generic method: InputReader.choose uses a generic type and Function to display selectable options.

17. Sample End-to-End Scenario

Example flow using only the program's supported concepts: a user enters Delhi as the starting point, Jaipur as the destination, selects Bus, and the built-in route catalogue supplies 280 km. The user then enters the number of travellers and nights. The system calculates the trip duration, prints Low/Medium/High ranges, and asks the user to choose a tier. It creates pre-departure, travel, arrival, destination-day, and final-day segments as applicable. The user enters actual spending for each category. If a category exceeds its planned maximum, the system prints an **OVER** alert and, when another segment remains, looks for cheaper suggestions for that category. At the end, it prints group planned spending, group actual spending, and actual per-person spending.

18. Strengths of the Current Implementation

- Clear end-to-end MVP workflow from planning to post-trip comparison.
- Separates per-person estimates from group totals, which is useful for group travel.
- Uses ranges and a buffer instead of pretending that travel costs are exact.
- Segment-based planning makes the expense feedback more actionable than a single trip total.
- Provides corrective suggestions after an over-budget category.
- Includes input validation and avoids many common CLI input errors.
- Uses modular Java classes, making later GUI/API/database integration more manageable.

19. Limitations of the Current MVP

- Route distances are **limited to a small hard-coded list**; unknown routes rely on user-entered approximate distance.
- Transport rates, speeds, overheads, budget assumptions, and suggestions are hard-coded and are not live market data.
- There is no persistent user account or database; runtime data is held in memory.
- The recommendation catalogue is small and mostly generic, with only limited city-specific entries.
- The system handles one-way travel only.
- The interface is command-line based and does not provide a GUI/mobile/web experience.
- The current intelligence is rule-based estimation and threshold logic rather than machine learning or an external AI model.
- Actual expenses are entered manually; the code does not import bank, wallet, receipt, or booking data.

20. Recommended Future Enhancements

Area	Possible enhancement
Route intelligence	Integrate a maps/routing API for distance and travel-time estimation.
Live pricing	Use current transport, accommodation, and activity price APIs where available.
Personalization	Learn user spending patterns and adjust budget assumptions using historical data.
AI assistant	Add natural-language trip planning, explanations, and adaptive suggestions.
Persistent storage	Add a database for users, trips, expenses, saved itineraries, and history.
GUI/web/mobile	Build a user interface with charts, dashboards, maps, and editable budgets.
Expense capture	Add receipt OCR, CSV import, or transaction integration.
Multi-way trips	Support return trips and multi-city itineraries.
Currency	Support multiple currencies and automatic conversion.
Alerts	Add notifications when spending crosses thresholds.
Recommendation engine	Use destination-specific and user-specific recommendations instead of mostly generic entries.

22. Technology Stack

- Language: Java.
- **Interface: Command-line interface (CLI).**
- Core libraries: java.util collections, Scanner, Locale, and Function.
- Data persistence: **none** in the supplied implementation.
- External APIs: **none** in the supplied implementation.
- Architecture: modular object-oriented single-process application.

23. Key Formulae and Decision Rules

- Trip days = nights + 1.
- Non-shared transport per-person cost = rate/km × distance + overhead.
- Shared car/cab per-person cost = (rate/km × distance + overhead) ÷ travellers.
- Travel hours = distance ÷ speed + 0.75 hours; flights add 3.0 hours instead.
- Transit meals = max(1, round(travel hours ÷ 6)).
- Budget point estimate = sum of transport, food, stay, activities, and local transport × 1.12.
- Budget range = point estimate ±15%.
- OVER = actual > planned maximum; TRENDING_HIGH = actual > 105% of planned midpoint while still inside the band; otherwise ON_TRACK.

24. Conclusion

The supplied Java program is a functional console-based MVP for one-way travel planning and expense management. **Its main contribution is the combination of budget-band selection, segment-wise cost planning, manual expense tracking, threshold-based feedback, and cheaper alternatives within a single workflow.** The implementation is intentionally self-contained and uses predefined assumptions, making it suitable as a prototype. To turn it into a production-grade intelligent travel platform, the **next development stage would need live route and pricing data, persistent storage, richer destination recommendations, automated expense capture, a user-facing GUI/web/mobile layer, and more adaptive personalization or AI capabilities.**